

# KOA COMPILER

## System Design Report

Architecture & Design Decision Analysis

Version: 0.1.0 (Experimental/Educational)

Date: May 31, 2026

License: MIT OR Apache-2.0

---

Repository: [github.com/riipandi/koa](https://github.com/riipandi/koa)

Rust Edition: 2024 | LLVM: 15 | inkwell: 0.4.0

# Table of Contents

|    |                                     |
|----|-------------------------------------|
| 1  | Executive Summary                   |
| 2  | Scope & Technical Goals             |
| 3  | Compiler Pipeline Architecture      |
| 4  | Architecture Decision Records (ADR) |
| 5  | Type System Design                  |
| 6  | Intermediate Representation (IR)    |
| 7  | Runtime & Memory Management         |
| 8  | Module System Design                |
| 9  | Error Handling Strategy             |
| 10 | Dependency Architecture             |
| 11 | Comparison with Other Languages     |
| 12 | Component Status Matrix             |
| 13 | Known Issues & Bugs                 |
| 14 | Technical Debt Register             |
| 15 | Test Coverage Analysis              |
| 16 | Risk Assessment                     |
| 17 | Security Considerations             |
| 18 | Performance Considerations          |
| 19 | Strengths, Trade-offs & Gaps        |
| 20 | Recommendations                     |
| A  | Appendix: ADR Details               |
| B  | Appendix: Contribution Guidelines   |

# 1. Executive Summary

---

Koa is a general-purpose compiled programming language designed as "TypeScript compiled to native code via LLVM". The compiler architecture follows a classic pipeline: Lexer, Parser, Type Checker, IR Lowering, and LLVM Codegen, organized into three workspace crates (koa, koa-cli, koa-runtime). The project adopts a design philosophy influenced by Rust (error handling, module system), Go (simplicity, runtime safety), and TypeScript (syntax familiarity).

Fifteen Architecture Decision Records (ADR) have been documented, covering decisions from naming conventions to memory safety models. The codebase totals approximately 7,918 lines of Rust with 83 passing test cases. This report provides comprehensive analysis of the system design, known issues, technical debt, and recommendations for project improvement.

## 2. Scope & Technical Goals

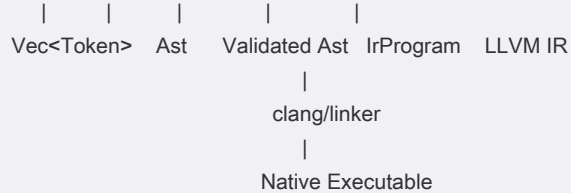
- TypeScript-familiar syntax for low learning curve
- Static typing with explicit type annotations
- Memory safety via concurrent garbage collector
- Fast performance compiled to native code via LLVM
- Simple module system with explicit imports/exports
- Built-in package manager with git-based dependencies
- FFI support via on-the-fly bindgen

### Implementation Status

| Component       | Status   | Lines | Tests |
|-----------------|----------|-------|-------|
| Lexer           | Complete | 451   | 4     |
| Parser          | Complete | 1,098 | 6     |
| AST Definitions | Complete | 519   | -     |
| Type Checker    | Complete | 1,087 | 30    |
| IR Lowering     | Complete | 963   | 2     |
| LLVM Codegen    | Partial  | 640   | 7     |
| Runtime (GC)    | Stub     | 80    | 0     |
| Runtime (Alloc) | Partial  | 91    | 0     |
| Runtime (I/O)   | Complete | 164   | 0     |
| CLI Tool        | Complete | 567   | -     |

## 3. Compiler Pipeline Architecture

Source (.koa) -> Lexer -> Parser -> Type Checker -> IR Lowering -> LLVM Codegen



### Stage 1: Lexer

Handwritten character-by-character scanner (451 lines, 13 functions). Produces Vec<Token> with Span metadata. Handles UTF-8, skips whitespace and comments (`//`, `///`, `/* */`).

### Stage 2: Parser

Recursive descent parser (1,098 lines, 51 functions). Uses operator precedence climbing. Error recovery via synchronization to semicolons/braces.

### Stage 3: Type Checker

Single-pass with scope tracking (1,087 lines, 29 functions). Supports structural typing for interfaces, monomorphization for generics, limited type inference.

### Stage 4: IR Lowering

SSA-like IR with Alloca/Store/Load pattern (963 lines, 25 functions). Basic blocks with named labels. Function specialization for monomorphization.

### Stage 5: LLVM Codegen

Uses inkwell 0.4.0 for LLVM 15 bindings (640 lines, 16 functions). Maps IrType to BasicTypeEnum. Handles external function declarations (C ABI).

### Runtime Component

Provides BumpAllocator, GarbageCollector (stub), and I/O FFI functions using C ABI. Linked with user programs via clang to produce final executables.

## 4. Architecture Decision Records (ADR)

Koa maintains 15 Architecture Decision Records documenting major design choices. Each ADR follows Status-Context-Decision-Rationale-Consequences format.

| ADR | Title                 | Influence  | Trade-off                       |
|-----|-----------------------|------------|---------------------------------|
| 001 | snake_case Naming     | Rust       | Familiarity vs convention       |
| 002 | Required Return       | TypeScript | Explicitness vs conciseness     |
| 003 | No Relative Paths     | Rust, Go   | Refactoring ease vs flexibility |
| 004 | JSON Lockfile         | npm        | Tooling vs TOML                 |
| 005 | GC Configuration      | Go         | Custom naming vs ecosystem      |
| 006 | No Arrow Functions    | Rust, Go   | Simplicity vs convenience       |
| 007 | Integer Enums         | C, Rust    | FFI vs expressiveness           |
| 008 | Rust Doc Comments     | Rust       | Markdown vs simplicity          |
| 009 | No OOP Inheritance    | Go         | Simplicity vs OOP               |
| 010 | Explicit Annotations  | TypeScript | Readability vs verbosity        |
| 011 | FFI via Bindgen       | Zig        | Automation vs control           |
| 012 | Structural Interfaces | Go         | Flexibility vs nominal          |
| 013 | Runtime Safety        | Go         | GC overhead vs simplicity       |
| 014 | Hybrid Imports        | Rust+TS    | Flexibility vs consistency      |
| 015 | Module Resolution     | Rust       | Clarity vs convenience          |

## 5. Type System Design

### Primitive Types

```
// Integers
let x: i8/i16/i32/i64/i128/usize = 42;
let y: u8/u16/u32/u64/u128/usize = 255;

// Floats
let x: f32 = 3.14; let y: f64 = 3.14159;

// Other
let b: bool = true; let s: string = "hello";
```

### Complex Types

```
struct Point { x: i32; y: i32; }
enum Color { Red, Green, Blue }
interface Printable { fn to_string(self): string; }
struct Container<T> { value: T; }
let arr: [10]i32 = [1, 2, 3, ...];
let ptr: *Point = alloc(Point);
let opt: ?i32 = null;
fn fn_ptr: fn(i32, i32) -> i32 = add;
const FileError = error { NotFound, AccessDenied };
```

### Generic Constraints

```
fn distance<T: Numeric>(p: Point<T>): T { ... }
fn process<T: Printable + Comparable>(item: T): T { ... }
// Monomorphization generates specialized versions
```

## 6. Intermediate Representation (IR)

### IR Instruction Set

| Category   | Instruction | Description             |
|------------|-------------|-------------------------|
| Memory     | Alloca      | Stack allocation        |
| Memory     | Store       | Write to pointer        |
| Memory     | Load        | Read from pointer       |
| Memory     | GEP         | Get element pointer     |
| Arithmetic | Binary      | Add, Sub, Mul, Div, Mod |
| Arithmetic | Unary       | Negation, logical not   |
| Arithmetic | Cmp         | Comparison operations   |
| Control    | Branch      | Conditional branch      |
| Control    | Jump        | Unconditional jump      |
| Control    | Return      | Return from function    |
| Function   | Call        | Function call           |

### IR Type Mapping

| Koa Type      | -> IrType         | -> LLVM Type    |
|---------------|-------------------|-----------------|
| i32           | -> Int32          | -> i32          |
| f64           | -> Float64        | -> double       |
| bool          | -> Bool           | -> i1           |
| string        | -> String         | -> i8*          |
| void          | -> Void           | -> void         |
| *T            | -> Pointer(T)     | -> T*           |
| [N]T          | -> Array(T, N)    | -> [N x T]      |
| struct {x, y} | -> Struct(T1, T2) | -> { T1, T2 }   |
| enum          | -> Enum{variants} | -> { i32, ... } |



## 7. Runtime & Memory Management

### Memory Management Architecture

```
Source Code -> LLVM IR -> Stack Allocation (Alloca)
               -> Heap Allocation (alloc) -> BumpAllocator
                                   -> GarbageCollector
                                       |-- Mark Phase
                                       |-- Sweep Phase
```

### BumpAllocator

Fast bump pointer allocator with alignment support. Tracks start, current, and end pointers. Returns None when full. Supports reset() for reuse.

### Garbage Collector (Planned)

Tri-color mark-sweep algorithm. Configuration via CLI flags (--gc-percent), env vars (KOA\_GC\_PERCENT), and programmatic API.

### Standard Library Architecture

1. Koa Declaration (library/std/io.koa):  
`pub extern fn println(s: string): void;`
2. LLVM Codegen: declare void @koa\_println(i8\*)
3. Rust Implementation (crates/koa-runtime/src/io.rs):  
`#[unsafe(no_mangle)]  
pub extern "C" fn koa_println(s: *const c_char) { ... }`
4. Linking: `clang user_code.o libkoa_runtime.a -o user_program`

## 8. Module System Design

### Import Styles

```
// Module-level import (Go/Rust style)
import from "std/io";
io.println("Hello");

// Specific item import (TypeScript style)
import from "std/io/println";
println("Hello");

// With alias
import from "std/io" as stdio;
```

### Resolution Order

- 1. stdlib (library/std/)
- 2. External dependencies (Koa.lock)
- 3. Workspace crates
- 4. Local src/ modules

### Local Module Structure

```
src/
main.koa      # Entry point
utils.koa     # File module
math/
  mod.koa     # REQUIRED for directory
  algebra.koa # Child module
```

## 9. Error Handling Strategy

---

### Compiler Error Handling

Uses `miette::Result<T>` consistently. `?` operator (415 usages) for propagation. Custom errors via `miette::miette!` macro. External errors via `.into_diagnostic()`?

### User Program Error Handling

```
const FileError = error { NotFound, AccessDenied };  
fn read_file(path: string): FileError!string {  
    let content: string = try fs.read(path);  
    return content;  
}  
  
fn safe_read(path: string): string {  
    return read_file(path) catch |err| {  
        println("Error: {}", err);  
        return "default";  
    };  
}
```

# 10. Dependency Architecture

| Dependency | Version | Purpose                    | Used By |
|------------|---------|----------------------------|---------|
| inkwell    | 0.4.0   | LLVM 15 safe Rust bindings | koa     |
| miette     | 5.10    | Rich error diagnostics     | koa     |
| clap       | 4.4     | CLI argument parsing       | koa-cli |
| tokio      | 1.35    | Async runtime (planned)    | koa-cli |
| serde      | 1.0     | Serialization              | koa-cli |
| colored    | 2.0     | Terminal colors            | koa-cli |
| which      | 6.0     | Find executables           | koa-cli |

## Release Profile

```
[profile.release]
lto = true      # Link-time optimization
codegen-units = 1 # Maximum optimization
strip = true    # Remove debug symbols
```

## 11. Comparison with Other Languages

| Feature        | Koa        | Go       | Rust       | Zig        |
|----------------|------------|----------|------------|------------|
| Memory Safety  | GC         | GC       | Borrow Ck  | Manual     |
| Type System    | Structural | Nominal  | Nominal    | Structural |
| Error Handling | ErrorUnion | Multiple | Result     | ErrorUnion |
| Generics       | Mono       | St Mono  | Mono+Trait | Comptime   |
| Module System  | Hybrid     | Package  | Mod+Use    | Import     |
| FFI            | C ABI      | CGo      | unsafe     | @cimport   |
| Syntax Base    | TS+Rust    | C-like   | Own        | C+Rust     |
| Learning Curve | Low-Med    | Low      | High       | Medium     |

## 12. Component Status Matrix

Detailed status of each component across build, test, documentation, and production readiness dimensions.

| Component       | Build   | Tests    | Docs    | Production       |
|-----------------|---------|----------|---------|------------------|
| Lexer           | OK      | 4 tests  | OK      | Complete         |
| Parser          | OK      | 6 tests  | OK      | Large (1098 LoC) |
| Type Checker    | OK      | 30 tests | OK      | Large (1087 LoC) |
| IR Lowering     | OK      | 2 tests  | Partial | Complete         |
| LLVM Codegen    | OK      | 7 tests  | Partial | Partial          |
| Runtime (GC)    | OK      | NONE     | OK      | STUB             |
| Runtime (Alloc) | OK      | NONE     | OK      | Basic only       |
| Runtime (I/O)   | OK      | NONE     | OK      | Complete         |
| CLI Tool        | OK      | NONE     | OK      | Complete         |
| Stdlib          | Partial | NONE     | OK      | Minimal          |
| CI/CD           | OK      | N/A      | N/A     | NEW              |

## 13. Known Issues & Bugs

### Critical Issues

| ID     | Issue                                 | Status | Impact                  |
|--------|---------------------------------------|--------|-------------------------|
| KI-001 | GC implementation is stub             | Open   | Runtime not functional  |
| KI-002 | String escape sequences not processed | Open   | Use puts() for newlines |

### Moderate Issues

| ID     | Issue                           | Status | Workaround             |
|--------|---------------------------------|--------|------------------------|
| KI-003 | unwrap() in production code (4) | Open   | Could panic            |
| KI-004 | Parser/TypeChecker >1000 LoC    | Open   | Maintenance difficulty |
| KI-005 | cargo audit broken (CVSS issue) | Open   | Manual review          |

### Resolved Issues

| ID     | Issue                            | Fixed In       |
|--------|----------------------------------|----------------|
| KI-006 | 7 unsafe pointer errors in io.rs | commit 8c3432a |
| KI-007 | No CI pipeline                   | commit 8c3432a |
| KI-008 | BumpAllocator alignment ignored  | commit 8c3432a |

## 14. Technical Debt Register

| Debt Item           | Category      | Impact              | Effort  | Priority      |
|---------------------|---------------|---------------------|---------|---------------|
| GC not implemented  | Architecture  | Runtime unusable    | 40+ hrs | P0 - Critical |
| ArenaAllocator stub | Code Quality  | Dead code           | 2 hrs   | P2 - Low      |
| 11 TODO items       | Code Quality  | Incomplete features | 20+ hrs | P2 - Medium   |
| No benchmarks       | Performance   | No baseline         | 4 hrs   | P2 - Medium   |
| Missing ADRs (5)    | Documentation | Decision gaps       | 6 hrs   | P3 - Low      |
| No fuzzing          | Quality       | Untested edge cases | 8 hrs   | P2 - Medium   |



# 15. Test Coverage Analysis

## Test Distribution

| Component     | Tests | Percentage | Coverage                              |
|---------------|-------|------------|---------------------------------------|
| Type Checker  | 30    | 36%        | Good - inference, constraints, errors |
| LLVM Generics | 7     | 8%         | Moderate - basic scenarios            |
| Interface     | 7     | 8%         | Moderate - satisfaction checking      |
| Parser        | 6     | 7%         | Low - basic parsing only              |
| Generics      | 8     | 10%        | Moderate - type parameters            |
| Enum Usage    | 5     | 6%         | Low - basic enum patterns             |
| Enum Tests    | 6     | 7%         | Low - basic enum lowering             |
| Lexer         | 4     | 5%         | Low - tokenization only               |
| IR            | 2     | 2%         | Very Low - minimal                    |
| LLVM Gen      | 2     | 2%         | Very Low - minimal                    |

## Missing Test Coverage

- Runtime (GC, Alloc) - 0 tests
- CLI commands - 0 tests
- Error recovery - 0 tests
- Edge cases (overflow, null) - 0 tests
- Performance benchmarks - 0 tests
- Integration tests (end-to-end) - 0 tests

## 16. Risk Assessment

| Risk                        | Likelihood | Impact   | Mitigation                |
|-----------------------------|------------|----------|---------------------------|
| GC blocks runtime           | High       | Critical | Adopt existing GC library |
| Files become unmaintainable | Medium     | High     | Refactor into modules     |
| LLVM upgrade breaks codegen | Low        | High     | Pin version, test suite   |
| No performance baseline     | Medium     | Medium   | Add benchmarks            |
| Unsafe code in runtime      | Low        | High     | Add miri tests            |
| Missing edge case handling  | Medium     | Medium   | Add fuzzing               |

## 17. Security Considerations

---

### Addressed

- FFI functions have null checks
- Unsafe code marked with safety docs
- BumpAllocator has alignment validation
- CI pipeline runs clippy with -D warnings

### Open Concerns

- No input validation for Koa source files (DoS potential)
- No sandboxing for FFI calls
- No dependency vulnerability scanning (cargo audit broken)
- No fuzzing for parser/typechecker
- No bounds checking in some runtime functions

## 18. Performance Considerations

---

### Current Bottlenecks (Estimated)

- Parser:  $O(n)$  recursive descent - Acceptable
- Type Checker:  $O(n^2)$  for nested generics - Potential issue
- IR Lowering:  $O(n)$  with HashMap lookups - Acceptable
- LLVM Codegen:  $O(n)$  instruction mapping - Acceptable
- Monomorphization:  $O(n \times m)$  for generic instantiations - Could bloat

### Missing Performance Data

- No compilation benchmarks
- No memory usage profiles
- No binary size analysis
- No startup time measurements

## 19. Strengths, Trade-offs & Gaps

### Architecture Strengths

- Clear Pipeline: Five-stage with well-defined contracts
- ADR Documentation: 15 documented decisions with rationale
- Error Handling: Consistent miette integration
- Module System: Hybrid import without wildcard complexity
- FFI Design: C ABI via Rust runtime
- Testing: 83 test cases covering key components

### Known Trade-offs

| Decision              | Trade-off        | Mitigation          |
|-----------------------|------------------|---------------------|
| No Borrow Checker     | GC overhead      | Race detector       |
| Explicit Annotations  | More verbose     | Better IDE support  |
| No Arrow Functions    | Verbose closures | fn already short    |
| Structural Interfaces | Type error debug | Clear messages      |
| Handwritten Parser    | More code        | Full control        |
| Monomorphization      | Binary bloat     | Acceptable for most |

## 20. Recommendations

### Architecture Health Score

| Aspect         | Score | Notes                              |
|----------------|-------|------------------------------------|
| Modularity     | 7/10  | Pipeline clear, modules too large  |
| Documentation  | 9/10  | Comprehensive ADRs                 |
| Testability    | 8/10  | 83 tests, uneven coverage          |
| Extensibility  | 8/10  | Pipeline easy to extend            |
| Error Handling | 9/10  | Miette integration excellent       |
| Code Quality   | 7/10  | Clippy clean, some unsafe          |
| OVERALL        | 8/10  | Solid architecture, needs maturing |

### Priority Refactoring

- Split parser/mod.rs (1098 LoC) into 4 modules
- Split typeck/checker.rs (1087 LoC) into 3 modules
- Implement GC or adopt library
- Add benchmark suite
- Document missing ADRs

### Missing ADRs to Document

- Error recovery strategy in parser
- Monomorphization vs dynamic dispatch
- IR design choices
- Build system integration
- Async runtime model

## 21. Appendix: ADR Details

---

### ADR-001: snake\_case Naming

Status: Accepted. Use snake\_case for variables/functions, PascalCase for types. Consistent with Rust, more readable for multi-word identifiers.

### ADR-009: No OOP Inheritance

Status: Accepted. Do not support class inheritance. Use composition + structural interfaces. Reduces complexity, avoids diamond problem, consistent with Go.

### ADR-013: Runtime Memory Safety

Status: Accepted. Runtime Safety with Race Detectors. GC for heap safety. ThreadSanitizer for data races. Encourages std/channel for communication.

### ADR-014: Hybrid Import System

Status: Accepted. Support both module-level and specific item imports. No wildcard imports. Balances Go-style prefixing with TypeScript convenience.

### ADR-015: Rust-style Module Resolution

Status: Accepted. Directory modules MUST have mod.koa. Clear resolution order: stdlib -> external -> workspace -> local. Consistent with Rust conventions.

## 22. Appendix: Contribution Guidelines

---

### Good First Issues

- Implement escape sequences in lexer
- Add tests for runtime components
- Split parser/mod.rs into smaller modules
- Document missing ADRs
- Add benchmarks for compilation pipeline

### Code Review Checklist

- `cargo clippy -- -D warnings` passes
- `cargo test --workspace` passes
- `cargo fmt --check` passes
- No new `unwrap()` in production code
- ADR updated if design decision changed
- Tests added for new functionality

### Development Workflow

- Run `cargo test` to ensure baseline passes
- Edit files, run `cargo build` frequently
- Use `cargo test test_name` for rapid iteration
- Run `cargo fmt` before committing
- Run `cargo clippy -- -D warnings` and fix all
- Final check: `cargo test && cargo clippy -- -D warnings`